

# SBI 812: Machine Learning in Bioinformatics

## Lecture 1: Introduction to Machine Learning and pandas

Duration: 3 hours Level: Introductory Practical theme: From a biological table to a trustworthy analysis-ready dataset

### 1. Why this lecture matters

Machine learning begins long before a model is trained. A useful analysis depends on understanding the biological question, recognising what each row and column represents, checking data quality, and avoiding information leakage. In this lecture, we introduce machine learning through a small gene-expression dataset and learn the pandas operations needed to inspect it. General Python programming is taught in SBI 811 and is therefore not repeated here.

The goal is not to memorise commands. The goal is to develop a repeatable habit:

*Ask a biological question, inspect the data, check assumptions, write reproducible code, and explain what the result means.*

### Learning outcomes

By the end of the lecture, a student should be able to:

1. Explain machine learning in plain language.
2. Distinguish supervised from unsupervised learning.
3. Distinguish classification, regression, clustering and dimensionality reduction.
4. Identify samples, features, targets and identifiers in a biological dataset.
5. Start the course computing environment and explain how pandas is imported.
6. Create, load, inspect, select and filter a pandas DataFrame.
7. Detect missing values and incorrect data types.
8. Produce simple summaries without overstating their biological meaning.
9. Save a cleaned table reproducibly.
10. Use AI as a coding assistant while independently checking its output.

## 2. Suggested lesson schedule

| Time      | Activity                                                        |
|-----------|-----------------------------------------------------------------|
| 0:00-0:20 | What machine learning is and why it is useful in bioinformatics |
| 0:20-0:45 | Samples, features, targets and major learning tasks             |
| 0:45-1:05 | Meeting pandas: imports, DataFrames and basic notation          |
| 1:05-1:15 | Break                                                           |
| 1:15-2:05 | Guided practical: opening and inspecting biological data        |
| 2:05-2:35 | Selection, filtering, missing values and summaries              |
| 2:35-2:50 | AI-assisted code audit                                          |
| 2:50-3:00 | Independent check and take-home task                            |

## 3. What is machine learning?

Machine learning is a collection of methods that learn patterns from data in order to make predictions, assign labels, discover structure, or reduce complex data to a simpler representation.

In conventional programming, a programmer supplies explicit rules:

`data + human-written rules → answer`

In supervised machine learning, we provide examples for which the answer is already known:

`labelled examples + learning algorithm → fitted model`  
`fitted model + new data → predicted answer`

For example, instead of writing a fixed rule for every expression pattern associated with drug response, we may train a model using samples whose responses are known. The model can then estimate the response of a new sample.

Machine learning does not automatically:

- prove that one biological factor causes another;
- compensate for biased or poorly collected data;
- understand the biological meaning of a column;
- guarantee useful predictions on a different population or laboratory;
- make preprocessing and experimental design unimportant.

Machine learning finds statistical patterns. Biological interpretation and validation remain the responsibility of the researcher.

## Bioinformatics examples

| Biological question                                      | Possible input features               | Output or aim                    | ML task                  |
|----------------------------------------------------------|---------------------------------------|----------------------------------|--------------------------|
| Is a tumour sample malignant or benign?                  | Gene-expression measurements          | Class label                      | Classification           |
| What drug concentration inhibits growth by 50%?          | Molecular and phenotypic measurements | Continuous value                 | Regression               |
| Are there previously unknown cell populations?           | Single-cell gene expression           | Groups of similar cells          | Clustering               |
| Can thousands of genes be represented in two dimensions? | Gene-expression matrix                | Lower-dimensional representation | Dimensionality reduction |
| Which protein family does a sequence resemble?           | Sequence-derived features             | Family label                     | Classification           |

## 4. The language of a machine-learning dataset

Most introductory machine-learning datasets are rectangular tables.

- A sample or observation is one row. Examples include a patient, tissue sample, cell or sequence.
- A feature is an input measurement used by a model. Examples include gene expression, sequence length or GC content.
- A target is the value a supervised model is asked to predict.
- A label is often used for a categorical target.
- An identifier names a sample but normally should not be used as a predictive feature.
- Metadata describes the sample, experiment or measurement.

Suppose each row represents one microbial culture:

| Column                 | Meaning                            | Possible role                   |
|------------------------|------------------------------------|---------------------------------|
| sample_id              | Unique culture identifier          | Identifier                      |
| species                | Organism studied                   | Metadata or categorical feature |
| treatment              | Control or drug exposure           | Metadata or categorical feature |
| gene_a, gene_b, gene_c | Normalised expression measurements | Numerical features              |
| growth_rate            | Observed growth per hour           | Continuous target               |
| response               | Sensitive or resistant             | Categorical target              |

The role of a column depends on the scientific question. If the question is “Can gene expression predict drug response?”, **response** is the target and the gene measurements are features. If the question changes, the roles may change.

### Important distinction: prediction is not explanation

A feature can improve prediction without being the biological cause of the outcome. It may be correlated with another factor, a laboratory batch, the sampling strategy or an unrecorded variable. Predictive importance must not automatically be reported as biological causation.

## 5. Four common types of machine-learning task

### 5.1 Supervised learning

The training data contain known targets.

#### Classification

The target is categorical, such as:

- resistant or sensitive;
- diseased or healthy;
- species A, B or C.

#### Regression

The target is numerical and continuous, such as:

- growth rate;
- patient age;
- drug response score;
- protein abundance.

### 5.2 Unsupervised learning

There is no target column supplied to the algorithm.

#### Clustering

The aim is to identify groups of similar samples. A cluster is a mathematical grouping, not automatically a new biological subtype.

#### Dimensionality reduction

The aim is to represent many measurements with fewer derived dimensions. This is useful for visualisation and identifying broad patterns. Principal component analysis (PCA) will be studied later in the course.

### Quick check

Classify each question:

1. Predict a patient's survival time from clinical measurements.
2. Separate bacterial isolates into resistant and sensitive groups using labelled examples.
3. Search for natural groups among unlabelled tumour expression profiles.
4. Represent the expression of 20,000 genes using a small number of components.

Expected answers: regression, classification, clustering, and dimensionality reduction.

## 6. Starting the practical environment

Open the course folder in Windows File Explorer. Right-click inside the folder (on some systems, hold Shift while right-clicking), choose Open in Terminal, and start or select an Ubuntu/WSL terminal profile. The exact wording can differ between Windows versions.

Check that the course environment and pandas are available:

```
python --version
python -c "import pandas as pd; print(pd.__version__)"
```

The prompt may begin with (base). This indicates that the Anaconda base environment is active, which is acceptable for this course.

Start JupyterLab:

```
jupyter lab
```

Create a Jupyter notebook named:

```
lecture01_pandas.ipynb
```

If JupyterLab is not available, the same code can be placed in a file named `lecture01_pandas.py` and run with:

```
python lecture01_pandas.py
```

## 7. Meeting pandas

The fundamentals of Python are covered in SBI 811. In SBI 812, we will concentrate on using pandas to work with biological tables. The pandas commands will be introduced slowly, and every new form of notation will be explained when it first appears.

### 7.1 What is pandas?

pandas is a Python library for working with labelled, rectangular data. It is particularly useful for datasets in which rows represent biological samples and columns represent measurements or metadata.

pandas is not automatically available merely because Python is running. Before using it, we must import it.

```
import pandas
```

This instruction has two parts:

- `import` tells Python to load a library so that its tools become available in the current notebook or script.
- `pandas` is the name of the library being loaded.

After using `import pandas`, we refer to the library by its full name:

```
pandas.read_csv("lecture01_bacterial_expression.csv")
```

The dot in `pandas.read_csv` can be read as “use `read_csv` from pandas.” The function `read_csv()` reads a comma-separated table.

### 7.2 What does import pandas as pd mean?

Most pandas users write:

```
import pandas as pd
```

The words `as pd` give pandas a short name, or alias, for the current notebook or script. They do not install a different library and do not change pandas itself. They simply allow us to type `pd` instead of `pandas`.

Therefore, these two commands mean the same thing:

```
# Using the full library name
pandas.read_csv("lecture01_bacterial_expression.csv")

# Using the conventional alias
pd.read_csv("lecture01_bacterial_expression.csv")
```

The second form works only after `import pandas as pd` has been run. The alias `pd` is used almost universally in pandas documentation and examples, so we will use it throughout this course.

If a notebook produces `NameError: name 'pd' is not defined`, the import cell may not have been run. Execute:

```
import pandas as pd
```

### 7.3 How to read pandas commands

At first, treat each pandas instruction as a sentence. For example:

```
df = pd.read_csv("lecture01_bacterial_expression.csv")
```

Read it from right to left:

1. `pd` refers to the pandas library.
2. `.read_csv(...)` asks pandas to read a CSV file.

3. The text inside quotation marks is the filename.

4. `df` = gives the resulting table the name `df` so that we can use it again.

The name `df` is short for `DataFrame`. A `DataFrame` is pandas' representation of a table with labelled rows and columns.

Another example is:

```
df.head(3)
```

This means “use the `head` method belonging to `df` and show the first three rows.” The value `3` is an argument supplied to the method. If no number is supplied, `df.head()` shows the first five rows.

To select a column, pandas commonly uses square brackets:

```
df["gene_a"]
```

Read this as “from `df`, select the column named `gene_a`.” The quotation marks indicate that `gene_a` is a column name, not another command.

You do not need to memorise all this notation immediately. Use the same pattern repeatedly, predict what a command should return, run it, and inspect the result.

## 8. Opening the lecture dataset with pandas

In most bioinformatics analyses, we do not begin by typing a dataset into pandas. The measurements already exist in a file produced by a laboratory instrument, database, collaborator or earlier analysis step. Our first pandas task is therefore to open an existing data file.

For this lecture, use the supplied file:

```
lecture01_bacterial_expression.csv
```

The file contains a small artificial bacterial gene-expression dataset. It is designed for learning and is not evidence about any real organism or treatment. The same file will be used for all examples and exercises in this lecture.

Place the CSV file in the same folder as the Jupyter notebook. Its first rows look like this:

| sample_id | species | treatment | gene_a | gene_b | gene_c | growth_rate | response  |
|-----------|---------|-----------|--------|--------|--------|-------------|-----------|
| S01       | E_coli  | control   | 5.1    | 2.0    | 7.2    | 0.91        | sensitive |
| S02       | E_coli  | drug      | 8.2    | 3.8    | 6.1    | 0.42        | resistant |
| S03       | E_coli  | control   | 5.5    | 2.2    | 7.0    | 0.88        | sensitive |

The extension `.csv` means comma-separated values. Each line represents one sample, and commas separate the columns. A CSV file stores the data but does not store Excel-style colours, formulas or multiple worksheets.

### 8.1 Import pandas

Run the import first:

```
import pandas as pd
```

### 8.2 Read the CSV file

```
expression_data = pd.read_csv("lecture01_bacterial_expression.csv")
```

Read this instruction as follows:

1. `pd` refers to pandas.
2. `read_csv()` tells pandas to read a comma-separated file.
3. `"lecture01_bacterial_expression.csv"` is the name of the file.
4. `expression_data` = gives the opened table a name that we can reuse.

The resulting object is a pandas DataFrame. The CSV file on disk has not been changed; pandas has read its contents into the current notebook session.

Display the DataFrame:

```
expression_data
```

In short examples, DataFrames are often named `df`. A descriptive name such as `expression_data` is easier to understand in a longer analysis.

If the file cannot be found, check the current working directory:

```
from pathlib import Path

print(Path.cwd())
print(list(Path.cwd().glob("*.csv")))
```

Do not repeatedly change paths until the error disappears. First identify where Python is working and where the file is stored.

## 9. Inspect before analysing

Never assume a file has loaded correctly merely because no error appeared.

### View rows

```
expression_data.head()
expression_data.head(3)
expression_data.tail(2)
```

### Check dimensions and column names

```
expression_data.shape
expression_data.columns
expression_data.index
```

`shape` returns `(number_of_rows, number_of_columns)`. For this dataset, the expected shape is `(12, 8)`.

### Check data types and missing values

```
expression_data.dtypes
expression_data.info()
expression_data.isna().sum()
```

Important questions include:

- Are numerical measurements stored as numbers?
- Are identifiers stored as text?
- Are any values missing?
- Did a column acquire the wrong name?
- Does the number of rows agree with the expected number of samples?

### Numerical summaries

```
expression_data.describe()
```

Include non-numerical columns if needed:

```
expression_data.describe(include="all")
```

Summary statistics are quality-control clues, not final biological conclusions. An unusual minimum or maximum may be a real observation, a unit problem, a data-entry error or a failed assay.

### Categorical summaries

```
expression_data["species"].value_counts()
expression_data["treatment"].value_counts()
expression_data["response"].value_counts()
```

Check unique values:

```
expression_data["species"].unique()
expression_data.nunique()
```

This can expose inconsistent spellings such as `control`, `Control` and `ctrl`, which pandas treats as different categories.

## 10. Selecting columns and rows

### Select one column

```
expression_data["gene_a"]
```

One column is usually a pandas Series.

```
type(expression_data["gene_a"])
```

### Select several columns

```
gene_columns = ["gene_a", "gene_b", "gene_c"]
expression_data[gene_columns]
```

Several columns form a DataFrame.

```
type(expression_data[gene_columns])
```

### Select by labels with `loc`

```
expression_data.loc[0:3, ["sample_id", "species", "gene_a"]]
```

With the default integer index, `loc[0:3]` includes both row labels 0 and 3.

### Select by position with `iloc`

```
expression_data.iloc[0:4, 0:3]
```

`iloc[0:4]` includes positions 0, 1, 2 and 3, but not 4.

### Why the distinction matters

`loc` works with labels; `iloc` works with integer positions. Confusing them can silently select the wrong samples after the index changes.

## 11. Filtering biological observations

Create a Boolean condition:

```
expression_data["treatment"] == "drug"
```

Use it to select rows:

```
drug_samples = expression_data.loc[
    expression_data["treatment"] == "drug"
].copy()

drug_samples
```

`.copy()` makes an independent DataFrame that can be modified safely.

### Multiple conditions

Use `&` for “and” and `|` for “or”. Put each condition in parentheses.

```
resistant_ecoli = expression_data.loc[
    (expression_data["species"] == "E_coli") &
    (expression_data["response"] == "resistant")
].copy()

resistant_ecoli
```

Filter a numerical measurement:

```
low_growth = expression_data.loc[
    expression_data["growth_rate"] < 0.50,
    ["sample_id", "species", "treatment", "growth_rate"]
]

low_growth
```

Always check the result:



```
print(low_growth.shape)
print(low_growth["sample_id"].tolist())
```

## 12. Missing values are biological and analytical decisions

The value for `gene_a` in sample S11 is missing.

Find rows with missing `gene_a` values:

```
expression_data.loc[expression_data["gene_a"].isna()]
```

Find complete values:

```
expression_data.loc[expression_data["gene_a"].notna()]
```

Possible responses to a missing value include:

- investigate the laboratory or data-processing record;
- repeat the measurement if possible;
- remove an affected sample or feature with justification;
- estimate the value using an appropriate imputation method;
- use a method that handles missing values.

Do not automatically use `dropna()` on the entire dataset. It can discard a large number of samples, change class balance and introduce bias.

For today's simple summary, calculate the mean using available values:

```
expression_data["gene_a"].mean()
```

By default, pandas ignores missing numerical values in this calculation. State that fact when reporting the result.

If a simple demonstration requires median imputation, preserve the original column:

```
clean_data = expression_data.copy()
gene_a_median = clean_data["gene_a"].median()
clean_data["gene_a_imputed"] = clean_data["gene_a"].fillna(gene_a_median)

clean_data[["sample_id", "gene_a", "gene_a_imputed"]]
```

This does not prove median imputation is best. In a real model, imputation must be learned from training data only; otherwise information from evaluation data can leak into model development.

## 13. Data types and conversion

Pandas commonly uses:

- `int64` for whole numbers;
- `float64` for decimal numbers and numerical columns containing missing values;
- `object` or `string` for text;
- `bool` for true/false values;
- `category` for a defined set of categories.

A common problem in biological data is that a column that should contain numbers is stored as text. Begin by checking the types inferred from the supplied CSV file:

```
expression_data.dtypes
```

Create a copy before changing types, then convert columns deliberately:

```
typed_data = expression_data.copy()

typed_data["sample_id"] = typed_data["sample_id"].astype("string")
typed_data["species"] = typed_data["species"].astype("category")
typed_data["gene_a"] = pd.to_numeric(
```

```

typed_data["gene_a"],
errors="coerce"
)

typed_data.dtypes

```

`errors="coerce"` changes values that cannot be converted into missing values. This is useful when a numerical column contains text such as `not_measured`, but it must not be used silently. Compare the missing-value counts before and after conversion:

```

print("Before conversion:", expression_data["gene_a"].isna().sum())
print("After conversion:", typed_data["gene_a"].isna().sum())

```

If the second count is larger, one or more non-numerical entries were converted to missing values and should be investigated. In this supplied dataset, both counts should equal 1 because `gene_a` already contains valid numbers and one documented missing value.

## 14. Creating new variables and grouped summaries

Create a copy of the original DataFrame, then add a simple indicator variable to the copy:

```

summary_data = expression_data.copy()
summary_data["low_growth"] = summary_data["growth_rate"] < 0.50
summary_data[["sample_id", "growth_rate", "low_growth"]]

```

Using a copy keeps `expression_data` unchanged for the exercises that follow.

Calculate treatment-level summaries:

```

expression_data.groupby("treatment")["growth_rate"].mean()

```

Calculate several summaries:

```

growth_summary = (
    expression_data
    .groupby("treatment")["growth_rate"]
    .agg(["count", "mean", "median", "std"])
)

growth_summary

```

Interpret cautiously. A difference between the two means does not by itself show that the treatment caused the difference. Species composition, sampling, measurement error and the small artificial sample all matter.

Create a two-way count table:

```

pd.crosstab(expression_data["treatment"], expression_data["response"])

```

Ask:

1. Is response balanced across treatments?
2. Could treatment reveal the response too directly in this artificial dataset?
3. Would this relationship be expected in a real prediction setting?
4. Are there variables that would not be available when predicting a new sample?

These questions anticipate data leakage, which occurs when information unavailable at prediction time is allowed to influence model training or evaluation.

## 15. A reproducible inspection checklist

Use this checklist whenever you receive a new biological table.

```

import pandas as pd

df = pd.read_csv("lecture01_bacterial_expression.csv")

print("Shape:", df.shape)
print("\nColumns:")
print(df.columns.tolist())
print("\nData types:")

```

```
print(df.dtypes)
print("\nMissing values:")
print(df.isna().sum())
print("\nFirst rows:")
print(df.head())
print("\nNumerical summary:")
print(df.describe())
```

Then ask:

- What does one row represent?
- Is there one row per independent biological sample?
- Which columns are identifiers, metadata, features and possible targets?
- Are units documented?
- Are categorical values consistent?
- Are any measurements impossible or implausible?
- Are repeated measures or technical replicates present?
- Could samples from the same patient or organism appear in both training and test data later?

## 16. Guided practical

Work individually for 15 minutes, then compare reasoning with a partner.

### Task A: inspect the dataset

Write code to answer:

1. How many samples and columns are present?
2. Which columns are numerical?
3. Which column contains a missing value, and which sample is affected?
4. How many samples belong to each species?
5. How many samples are labelled resistant?

### Task B: select and filter

Create tables containing:

1. only `sample_id` and the three gene columns;
2. only control samples;
3. only resistant *B. subtilis* samples;
4. samples with `gene_b` greater than 3.5;
5. the first five rows and first four columns using `iloc`.

### Task C: summarise

Calculate:

1. the median of each gene;
2. mean growth rate by species;
3. mean growth rate by treatment;
4. counts for every species-response combination.

### Task D: reason biologically

Discuss:

1. Which columns could be used to predict `response`?

2. Should `sample_id` be included as a feature? Why or why not?
3. If `treatment` is almost equivalent to `response`, what problem could occur?
4. Does the missing `gene_a` measurement justify deleting sample S11?
5. What additional metadata would be useful in a real experiment?

## 17. Suggested solutions

Try the tasks before opening this section.

```
# Task A
print(expression_data.shape)
print(expression_data.select_dtypes(include="number").columns.tolist())
print(expression_data.loc[expression_data.isna().any(axis=1)])
print(expression_data["species"].value_counts())
print((expression_data["response"] == "resistant").sum())

# Task B
genes_only = expression_data[["sample_id", "gene_a", "gene_b", "gene_c"]]

controls = expression_data.loc[
    expression_data["treatment"] == "control"
].copy()

resistant_bsubtilis = expression_data.loc[
    (expression_data["species"] == "B_subtilis") &
    (expression_data["response"] == "resistant")
].copy()

high_gene_b = expression_data.loc[expression_data["gene_b"] > 3.5].copy()

first_block = expression_data.iloc[0:5, 0:4]

# Task C
gene_medians = expression_data[["gene_a", "gene_b", "gene_c"]].median()

growth_by_species = expression_data.groupby("species")["growth_rate"].mean()

growth_by_treatment = expression_data.groupby("treatment")["growth_rate"].mean()

species_response_counts = pd.crosstab(
    expression_data["species"],
    expression_data["response"]
)
```

Task D does not have one-line answers. A good response should distinguish identifiers from predictive measurements, recognise leakage, avoid deleting data without investigation, and request experimental metadata such as batch, replicate, collection time, instrument and measurement units.

## 18. AI-assisted programming activity

AI use is permitted in this course, but every student remains responsible for the code and scientific claims submitted.

### A useful request to an AI assistant

I am a beginning Python student. Explain this pandas expression line by line:

```
df.loc[(df["species"] == "E_coli") &
       (df["growth_rate"] < 0.5),
       ["sample_id", "growth_rate"]]
```

Then give me a two-row example and show the expected result.  
Do not introduce commands that are not required.

### Verification procedure

For AI-generated code:

1. Explain what each line is intended to do.
2. Run it on a small dataset whose answer you can determine manually.

3. Inspect `shape`, columns, data types and missing values before trusting the result.
4. Compare the result with the biological question.
5. Check official documentation when behaviour is uncertain.
6. Record meaningful AI assistance according to course policy.

## Code-audit exercise

An AI assistant proposes:

```
df = pd.read_csv("lecture01_bacterial_expression.csv")
df = df.dropna()
X = df.drop(columns="response")
print("The data are clean and ready for machine learning.")
```

Identify at least five problems or unanswered questions.

Possible observations:

- `dropna()` removes rows without investigating why data are missing.
- The code does not report which samples were removed.
- `sample_id` remains among the possible model inputs.
- Text columns still require appropriate handling before most models can use them.
- The code does not define the prediction question.
- It does not separate features and target based on scientific reasoning.
- It does not check duplicated samples, incorrect types or implausible values.
- It does not address leakage, experimental batches or related samples.
- "Clean" is asserted without evidence.

## Privacy reminder

Do not upload confidential patient information, unpublished sensitive data, credentials, access tokens or restricted datasets to a public AI service. Use artificial or approved data and follow institutional requirements.

## 19. Independent formative check

Complete this section without AI or help from another person. It is a diagnostic exercise, not a punishment.

Using the same `expression_data` DataFrame, write code to:

1. display the first two rows;
2. report its shape;
3. count missing values;
4. select only drug-treated samples;
5. calculate the mean `gene_a` value using the available measurements;
6. explain why `sample_id` should not normally be a model feature.

## 20. Take-home exercise

Create a notebook named `lecture01_takehome.ipynb`.

1. Load the supplied file `lecture01_bacterial_expression.csv` with pandas.
2. Add a column called `experiment_batch` containing `batch_1` for the first six samples and `batch_2` for the last six.

3. Produce a table containing only drug-treated samples with `gene_a`, `gene_b`, `gene_c` and `response`.
4. Report missing values in that table.
5. Calculate mean gene expression by treatment.
6. Save the selected table as `drug_samples.csv` without an index column.
7. In 150-250 words, explain:
  - what one row represents;
  - a possible classification question;
  - a possible regression question;
  - one risk of misleading analysis;
  - how any AI assistance was checked.

## Submission standard

The notebook must run from top to bottom, contain explanatory Markdown, show relevant outputs and avoid unexplained copied code.

## 21. Key terms

| Term                     | Meaning                                                                                                  |
|--------------------------|----------------------------------------------------------------------------------------------------------|
| Algorithm                | A defined computational procedure                                                                        |
| Model                    | A fitted mathematical representation learned from data                                                   |
| Sample                   | One observation, usually represented by a row                                                            |
| Feature                  | An input variable used for learning or prediction                                                        |
| Target                   | The value a supervised model is trained to predict                                                       |
| Classification           | Prediction of a categorical target                                                                       |
| Regression               | Prediction of a numerical target                                                                         |
| Clustering               | Grouping observations without supplied labels                                                            |
| Dimensionality reduction | Representing many variables using fewer derived dimensions                                               |
| DataFrame                | A labelled two-dimensional pandas table                                                                  |
| Missing value            | A measurement that is absent or unavailable                                                              |
| Data leakage             | Use of information during model development that would not legitimately be available for new predictions |
| Reproducibility          | Ability to repeat an analysis from documented data and code                                              |

## 22. Summary

Machine learning in bioinformatics is not simply the act of calling a modelling function. It begins with a precise biological question and a trustworthy understanding of the data. In this lecture, we treated rows as biological samples, distinguished features from targets and identifiers, and used pandas to inspect, select, filter, summarise and save a table. These operations will support every later lecture.

Before the next class, you should be able to open a biological CSV file and answer:

1. What does one row represent?
2. What does each column mean?
3. Which columns are features, targets, identifiers and metadata?
4. Are any values missing or stored using the wrong type?

5. What evidence supports the claim that the table is ready for analysis?

## 23. Further reading

- [pandas: Getting started](#)
- [pandas read\\_csv documentation](#)
- [pandas DataFrame documentation](#)
- [scikit-learn: An introduction to machine learning](#)
- [scikit-learn: Common pitfalls and recommended practices](#)